pulp_soc: sequential area breakdown

LibreLane synthesis run `runs/syn_03` | SKY130 (sky130_fd_sc_hd, TT 25 °C 1.8 V) | flattened netlist | Prepared 30 Oct 2026

232,611
standard cells

3.12 mm²
standard-cell area (excl. black boxes)

1.30 mm² (41.6%)
flip-flop area

~38%
of all flops are in the HyperBus controller

1. Summary

- Synthesis of `pulp_soc` produced about **50,000 flip-flops** and **2,148 latches**. The flip-flops alone take **41.6%** of the standard-cell area (latches add another 1%), which is high for a SoC whose large memories (L2, ROM) are already black boxes.
- No large memory was accidentally turned into flip-flops. The flop count comes from real logic blocks, mostly **FIFOs and register banks in peripherals**.
- The **HyperBus controller** alone accounts for **~18,800 flops (~38%)**. Most of it is 8 copies of a transaction FIFO (`N_CH_HYPER = 8`).
- Four blocks (HyperBus, I2S PDM, FC HWPE, debug module) hold **~55%** of all flops. Each one is a possible area reduction, but every change is an architecture decision for the team and ADI.

2. Sequential cells in the netlist

Cell	Type	Count	Area (µm²)
`sky130_fd_sc_hd__dfrtp_2`	D flip-flop, async reset	44,944	1,180,913
`sky130_fd_sc_hd__dfxtp_2`	D flip-flop, no reset	3,472	73,851
`sky130_fd_sc_hd__dfstp_2`	D flip-flop, async set	1,578	41,462
`sky130_fd_sc_hd__dlxtp_1`	Latch, active high	2,049	30,764
`sky130_fd_sc_hd__dlxtn_1`	Latch, active low	99	1,486
Flip-flops total		**49,994**	**1,296,226 (41.6%)**
Latches total		**2,148**	**32,250 (1.0%)**

Source: `06-yosys-synthesis/reports/stat.rpt`. Black boxes not included in area: 6× `tc_sram` (L2), 1× `generic_rom`, 3× `gf22_FLL`.

3. Method and accuracy

> The design is flattened, so `stat.rpt` only reports the top level. The final netlist (57 MB) could not be transferred for this analysis. Flop counts per block were therefore **estimated from the Yosys synthesis log** (`yosys-synthesis.log`): every register Yosys reports while optimizing, with its hierarchical name and bit width.

- Coverage: **40,040 of 49,994 flip-flops (~80%)** could be attributed to a block. The rest are registers Yosys did not mention in the log.
- Percentages below are relative to all **49,994** flip-flops, so they are **lower bounds**.
- For exact numbers, run the script in Appendix A on the netlist (it takes a few seconds).

4. Flip-flops per block (ranked)

#	Block / hierarchy	Flops (est.)	% of all	
1	**HyperBus (uDMA)** `soc_peripherals_i.i_udma.i_hyper` Off-chip HyperRAM / HyperFlash controller, 8 command channels	18,797	37.6%	
2	**I2S PDM filter** `soc_peripherals_i.i_udma.i_i2s_udma.i_i2s_txrx.i_pdm` Decimation filter for digital (PDM) microphones	3,076	6.2%	
3	**FC HWPE accelerator** `fc_subsystem_i.fc_hwpe_gen` MAC accelerator next to the FC core (USE_HWPE=1)	3,055	6.1%	
4	**Debug module** `i_dm_top` halted_q / resuming_q: 1 bit per hart, 1024 harts	2,615	5.2%	
5	**AXI crossbar** `i_soc_interconnect_wrap.i_soc_interconnect` SoC interconnect (AXI xbar + interleaved/contiguous xbars)	1,757	3.5%	
6	**axi64_to_lint32** `i_soc_interconnect_wrap.i_axi64_to_lint32` Bridge for 64-bit AXI traffic coming from the cluster	1,437	2.9%	
7	**FC core (RI5CY)** `fc_subsystem_i.FC_CORE` 640 bits of these are PMP registers	1,296	2.6%	
8	**uDMA core** `soc_peripherals_i.i_udma.i_udmacore` Channel/address/size registers for all uDMA channels	1,057	2.1%	
9	**uDMA filter** `soc_peripherals_i.i_udma.i_filter` Data-processing filter unit in the uDMA	1,050	2.1%	
10	**Advanced timer** `soc_peripherals_i.i_apb_adv_timer` 4 PWM/timer channels + APB interface	980	2.0%	
11	**CDC FIFOs** `axi_slave_cdc_i / axi_master_cdc_i` Clock-domain-crossing FIFOs to/from the cluster	854	1.7%	
12	**uDMA SPI / I2C / SDIO / CAM / UART** `soc_peripherals_i.i_udma.*` Smaller uDMA peripherals combined	1,823	3.6%	
13	**AXI to AXI-Lite / APB** `i_soc_interconnect_wrap.i_axi_*` Protocol bridges to the peripheral bus	498	1.0%	
14	**SoC ctrl + event gen + GPIO + misc** `soc_peripherals_i.*` APB control registers, event generator, GPIO, timer unit	1,214	2.4%	
15	**Other (JTAG, I2S core, FC misc)** Remaining small blocks	530	1.1%	

5. Hotspot details

5.1 HyperBus controller — ~18,800 flops

Instance `soc_peripherals_i.i_udma.i_hyper.udma_hyperbus_i`. The number of command channels is set in `asic/pulp_soc/rtl/udma_subsystem.sv`, line 134: `localparam N_CH_HYPER = 8;`. Each channel has its own transaction FIFO and register interface.

Part	Instance	Flops (est.)
twd_tran_fifo (x8 channels)	`genblk1[0..7].u_cmd_if.twd_tran_fifo_i`	13,528
Per-channel register interface (x8)	`genblk1[0..7].u_cmd_if.u_reg_if`	1,720
Transaction CDC buffer	`u_dc_tran.u_din.buffer`	1,720
TX / RX / to-uDMA CDC buffers	`u_dc_tx, u_dc_rx, u_dc_toduma`	844
Global register interface	`u_reg_if`	200
FIFO, splitter, unpack, PHY, ctrl	`u_fifo, hyper_twd_trans_spliter_i, ...`	785

5.2 I2S PDM filter — ~3,100 flops

`i_i2s_udma.i_i2s_txrx.i_pdm`: filter for PDM digital microphones (CIC/decimation stages). Only needed if PDM microphones are part of the target application.

5.3 FC HWPE accelerator — ~3,100 flops

`fc_subsystem_i.fc_hwpe_gen.i_fc_hwpe` (streamer ~2,100, control ~950). Enabled by `USE_HWPE=1` in `SYNTH_PARAMETERS` of `config.yaml`. The cluster has its own accelerator path, so this one may not be needed.

5.4 Debug module — ~2,600 flops

`i_dm_top.i_dm_mem.halted_q` and `resuming_q` are 1,024 bits each: one bit per possible hart. PULP encodes the cluster ID in the hart ID, so the debug module is sized for 1,024 harts. It can be reduced, but this affects debugger support, so it is the lowest-priority option.

5.5 Interconnect, cluster bridge and FC core

AXI crossbar (~1,750), `axi64_to_lint32` buffers (~1,450) and CDC FIFOs (~850) are needed to connect the cluster and should stay. In the FC core, **640 bits are PMP registers** (physical memory protection), which could be disabled by parameter if not needed.

6. Options for discussion

Option	Est. flops saved	Impact / risk	Decision
Reduce `N_CH_HYPER` from 8 to 1	~13,000	Keeps HyperRAM support with a single command channel. Software (pulp-runtime) uDMA channel IDs must match.	Team + ADI
Remove HyperBus completely	~18,800	No external HyperRAM/Flash. uDMA channel/peripheral IDs change, so runtime and regression tests need updating.	Team + ADI
Set `USE_HWPE=0` (FC side only)	~3,100 + logic	Simple parameter change. Removes the FC accelerator only, not the cluster HWPE.	Team
Remove I2S PDM filter	~3,100	Only if PDM microphones are not needed. Needs an RTL change in the I2S block.	Team
Disable PMP in FC core	~640	Small gain. Check whether the boot code or runtime uses PMP.	Team
Reduce debug-module hart count	~2,000	Affects debug of cluster cores. Not recommended now.	Keep

> **Before changing any RTL:** every option above changes the hardware that software sees. Re-run the functional regression tests (`regression_tests`) after each change, and update `pulp-runtime` if uDMA channel or peripheral IDs move. Keep all edits in `asic/pulp_soc/rtl/` so Bender re-checkouts do not overwrite them.

7. Recommended next steps

- Run the script in Appendix A on `runs/syn_03/final/nl/pulp_soc.nl.v` to confirm the exact numbers.
- Decide with the team and ADI which peripherals the final chip needs (HyperBus, PDM, FC HWPE). Add this question to the L2/ROM/FLL email.
- Apply agreed changes, re-run synthesis, and compare against this baseline (3.12 mm², 41.6% sequential).

Appendix A — exact count script

Usage: `python3 ff_by_module.py runs/syn_03/final/nl/pulp_soc.nl.v 3` (last argument = hierarchy depth). It groups every flip-flop and latch by the name of the net on its Q pin.

```
#!/usr/bin/env python3
"""Count flip-flops / latches per hierarchy in a flattened Yosys netlist.

Usage:  python3 ff_by_module.py runs/syn_03/final/nl/pulp_soc.nl.v [depth]
Groups each sequential cell by the hierarchical name of the net on its Q pin.
"""
import re, sys, collections

nl = sys.argv[1]
depth = int(sys.argv[2]) if len(sys.argv) > 2 else 3
cell_re = re.compile(r'^\s*(sky130_fd_sc_\w+__(?:df|dl|sdf|edf)\w*)\s+\S+\s*\(', re.S)
q_re = re.compile(r'\.Q(?:_N)?\s*\(\s*(\\\S+|[^\s)]+)')

counts, kinds = collections.Counter(), collections.Counter()
text = open(nl, errors="ignore").read()
for stmt in text.split(';'):
    m = cell_re.match(stmt)
    if not m:
        continue
    kinds[m.group(1)] += 1
    q = q_re.search(stmt)
    name = q.group(1).lstrip('\\') if q else '<no Q>'
    name = re.sub(r'\[\d+\]$', '', name)
    if name.startswith('_') or name.startswith('<'):
        key = '<anonymous net>'
    else:
        key = '.'.join(re.sub(r'\[\d+\]', '[*]', name).split('.')[:depth])
    counts[key] += 1

total = sum(counts.values())
print(f"total sequential cells: {total}")
for k, v in kinds.most_common():
    print(f"  {k:32s} {v}")
print(f"\n{'count':>8} {'%':>6}  hierarchy (depth={depth})")
for k, v in counts.most_common(60):
    print(f"{v:8d} {100*v/total:6.1f}  {k}")
```